1 Introduction

In this class, we designed an in order pipelined processor and were introduced to how basic programs run on a CPU. In order processors can lack in performance as many times, instructions can take long times to execute, whether this is waiting on memory or another dependency. In this case, being able to execute instructions out of order is advantageous. Modern processors rely heavily on out-of-order execution to achieve high performance while efficiently utilizing hardware resources. Rather than executing instructions strictly in program order, out-of-order processors dynamically schedule instructions based on operand availability and resource constraints, allowing independent instructions to execute in parallel, reducing stalls caused by long-latency operations such as memory accesses and branch mispredictions. This project allowed us to understand fundamental computer architecture concepts such as register renaming, speculative execution, and memory hierarchy.

In this report, we provide an overview of the processor architecture and project goals. Next, we describe the evolution of the processor across each project checkpoint. We then discuss the advanced architectural features added to the processor, along with their associated trade-offs and performance impact. Finally, we conclude with observations about the final processor design and lessons learned throughout the process.

2 Project Overview

We designed and implemented an out-of-order RISC-V RV32IM processor in SystemVerilog. The processor follows the RISC-V instruction set architecture (ISA), meaning that all instructions must execute correctly while preserving the architectural behavior defined by the specification. Although instructions may execute out of order internally to improve performance, the processor must still maintain correct program behavior and commit instructions in-order.

Across the different checkpoints, we implemented the front end of the pipeline, register renaming, reservation stations, reorder buffer (ROB), physical register file, and execution and commit logic necessary for out-of-order operation. We also added support for branches, loads, and stores while ensuring proper hazard handling and pipeline flush recovery. Finally, we implemented several advanced architectural features to further optimize performance, including a split load-store queue with store forwarding, a SRAM-backed Gshare branch predictor, a Branch Target Buffer (BTB), a Return Address Stack (RAS), stride and next-line prefetchers, and a post-commit store buffer. At each checkpoint, we tested and debugged heavily through running programs and analyzing waveform behavior with synopsys VCS tools.

3 Design Description

3.1 Overview

This design was made incrementally with four main checkpoints, with each checkpoint introducing additional architectural complexity and functionality. The initial stage focused on constructing the front end of the pipeline, including instruction fetch integrated with an i-cache, an adapter to deserialize DRAM input, and an instruction queue to hold multiple fetched instructions. Once the front end was functional, we implemented the core out-of-order execution engine by adding register renaming, a physical register file, reservation stations, a reorder buffer (ROB), and issue and commit logic to support speculative and parallel instruction execution while maintaining precise architectural state. For the last checkpoint, we expanded support for control flow and memory operations through branch handling, branch recovery mechanisms, and a unified load-store queue capable to support memory operation. In the final stages of development, we incorporated several advanced performance optimizations, including a SRAM Gshare branch predictor, Branch Target Buffer (BTB), Return Address Stack (RAS), split load/store queues, stride and next-line prefetchers, and a post-commit store buffer. Together, these components allowed the processor to improve instruction throughput, reduce pipeline stalls, and better exploit instruction and memory-level parallelism.

3.2 Milestones

3.2.1 Checkpoint 1

During Checkpoint 1, we focused on designing the high-level microarchitecture and implementing the front-end fetch system for our out-of-order processor. We created a complete datapath diagram and finalized our choice of an ERR-style architecture. A parameterizable queue module was implemented to hold incoming instructions, while a cacheline adapter was developed to interface the instruction cache with the DRAM model. The main focus of the checkpoint was building the instruction fetch stage and integrating it with the instruction cache and memory subsystem. To achieve a fetch throughput of one instruction per cycle within the same cacheline, we implemented a 256-bit linebuffer that stored an entire cacheline and allowed sequential instructions to be read directly from the buffer instead of repeatedly accessing the cache. The fetch stage initialized the program counter to address 0xAAAAA000 and fed fetched instructions into the processor queue for the next stage to decode it.

This program was tested mainly through Verdi. We used the given ooo_test.s file and ensure that the instruction queue could be seen filling up. We would then assert the dequeue signal through a testbench and ensure the queue would fill up again.

3.2.2 Checkpoint 2

Checkpoint 2 introduced the full out-of-order execution units for integer arithmetic, multiplication, and division, but excluded branches, memory operations, and other control instructions. The work was divided three ways: the decode/rename front-end, the dispatch and ROB back-end, and the issue/execute units. All components were implemented in dedicated files and integrated in a single top-level module `cp2.sv`.

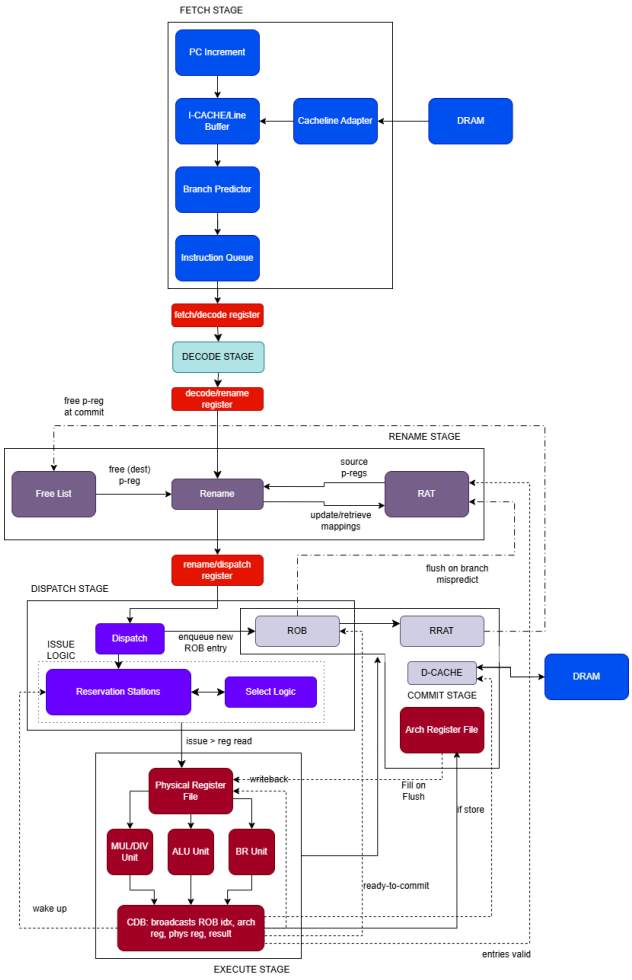

Figure 1: Checkpoint 2 block diagram.

The rename stage combined a Register Alias Table (RAT) and a free list. On each dispatch, the RAT translated architectural source registers to physical ones and the free list allocated a fresh physical destination register. The RAT flushed to the Retirement RAT (RRAT) snapshot on a pipeline squash so that architectural state could always be recovered. The dispatch stage sat between rename and the reservation stations: it simultaneously enqueued an entry into the ROB and routed the instruction to the correct reservation station (ALU, MUL, or DIV) based on the decoded operation type. A stall was generated whenever

the ROB was full, the free list was empty, or any target reservation station was full.

The issue stage held separate reservation station arrays for ALU, MUL, and DIV operations. Each cycle, each station scanned its entries for the oldest instruction whose physical source operands were marked ready in the scoreboard and whose functional unit was available, then issued it to the execute stage. A physical register scoreboard (`preg_lookup_table`) tracked readiness: a register was cleared (marked not-ready) at rename time and set again when a matching CDB broadcast arrived. The issue-to-execute handoff used a registered pipeline stage (`issue_execute_r`) to break the combinational path from reservation station to functional unit.

The execute stage contained three pipelined functional units. The ALU resolved all integer arithmetic and logic operations in a single cycle. The multiplier unit computed 32-bit products over multiple cycles, signaling availability when idle. The divider used a `DW_div_seq` IP with a one-cycle `start` pulse and a `complete` handshake. All three units broadcast results on a shared five-slot CDB bus (`cdb_bus[0--2]`; slots 3–4 were reserved for branches and memory). Each broadcast carried the destination physical register and result value; the ROB and all reservation stations listened on every slot to update operand readiness and capture results.

The commit stage retired the oldest ROB entry each cycle when its result was valid. On commit, the RRAT was updated with the destination mapping, the old physical register was returned to the free list, and the RVFI signals were driven from the committed entry. An incrementing 64-bit order counter tracked the architectural retire sequence for RVFI compliance.

Testing for CP2 relied on directed assembly programs that exercised RAW hazards, WAW sequences, long-latency divides, and back-to-back multiplications. We also wrote some division and multiplication edge cases to ensure that the IPs were properly implemented into the design.

3.2.3 Checkpoint 3

Checkpoint 3 completed the processor by adding full memory and branch support. The four main additions were: a unified load-store queue (LSQ), a D-cache with an access queue, a DRAM arbiter to share the memory bus between the I-cache and D-cache, and a static not-taken branch predictor alongside the necessary flush updates to all previous modules.

Our original plan was to implement a split LSQ with independent load and store queues. We however encountered ordering and disambiguation bugs that were difficult to isolate in the split design. Consequently we switched to a unified LSQ near the deadline in which loads and stores share a single ordered structure. The unified design enforced memory ordering naturally at the cost of memory-level parallelism that we later recovered in the advanced feature phase with a true split LSQ.

Additionally, we implemented the D-cache, a 2-way set-associative cache. Because loads and stores can both become ready in the same cycle, we introduced a small D-cache access queue to serialize requests. The queue holds the address, write mask, and data for each pending operation and drains one entry per cycle into the cache, decoupling the LSQ from

cache timing and preventing structural conflicts between the ROB and the LSQ.

Since both the I-cache and D-cache require access to the shared 64-bit DRAM bus for cache-line fills, we implemented a `dram_arbiter` that multiplexes between the two caches, granting the bus to the D-cache on demand and falling back to the I-cache otherwise. The arbiter ensures that an in-flight cache-line transfer is never interrupted mid-transfer, avoiding data corruption across the 256-to-64-bit cacheline-adapter boundary.

For CP3 we also implemented a static not-taken branch predictor. All branches are predicted not taken and the fetch PC advances sequentially. When the branch unit resolves a taken branch at execute time, the ROB raises a flush and restores the correct PC. Propagating flush correctly to every pipeline stage was the most time-consuming part of the checkpoint besides debugging since each module required explicit flush handling to clear speculative state without corrupting committed architectural state. Testing for this checkpoint used the given benchmark programs to confirm that all speculative instructions between the mis-fetched PC and the resolved target were squashed cleanly. We would run each simulation until the simulation halted or started hanging, after which we would start a round of debugging. We found that using numerous **$display** statements was quite helpful here as we could easily check the processor state in the simulation.log file without opening up Verdi.

This checkpoint also led to the encounter of many synthesis issues. Our area for this checkpoint was around 450k, which was already much higher than the allowed area for advanced features. Additionally, our critical path through the unified LSQ was incredibly long, requiring a 15ns clock to pass timing. This would eventually cause us several issues when implementing advanced features.

3.3 Advanced Design Features

3.3.1 Bimodal + Gshare + Tournament Predictor

Design To start our advanced features, we implemented a tournament-style branch predictor that combines a Bimodal predictor, a GShare predictor, and a Branch Target Buffer (BTB) to improve instruction fetch accuracy and reduce costly pipeline flushes. The Bimodal predictor learns the behavior of individual branches using a table of 2-bit saturating counters indexed by the program counter (PC), making it effective for branches with repetitive local behavior. The GShare predictor improves upon this by XORing the branch PC with a Global History Register (GHR), which stores the outcomes of recent branches. This allows the predictor to capture global branch patterns and correlations between branches. A separate chooser table acts as the tournament predictor and dynamically decides whether the Bimodal or GShare predictor is more reliable for a particular branch. During instruction fetch, both predictors generate a taken/not-taken prediction, and the chooser selects the final direction prediction. If the branch is predicted taken, the BTB supplies the predicted target address so the processor can immediately begin fetching from the predicted destination instead of waiting for branch execution. The predictor is updated at commit time using the actual branch outcome, ensuring only correct architectural information trains the predictor. We implemented the design using parameterizable predictor tables, 2-bit saturating

counters, a tagged BTB structure, and metadata snapshots that flow through the reorder buffer to allow correct predictor updates and recovery after mispredictions.

Testing Initially the processor was designed with a static not taken branch predictor. This means that branches are always not taken, but in the execute stage once we know if the branch should be taken or not, the pipeline is flushed and the PC is set to the calculated value. This performs poorly for loops as branches must be taken over and over again. Here is a comparison of the initial misprediction rate, compared to the Tournament predictor with both Gshare and Bimodal implemented.

Benchmark	**IPC (baseline static-NT)**	**IPC (Tournament + GShare + BTB)**
coremark_im	0.271	0.430
fft	0.379	0.452
mergesort	0.312	0.346
aes_sha	0.355	0.358
compression	0.308	0.513
image	0.293	0.316

Table 1: IPC comparison between the baseline static-not-taken predictor and the advanced tournament branch predictor incorporating GShare, Bimodal, and BTB structures.

Performance Analysis The addition of the GShare, Bimodal, BTB, and tournament branch predictor significantly improved IPC across nearly all benchmark workloads by reducing costly branch mispredictions and improving frontend fetch accuracy. Compared to the baseline static-not-taken predictor, the advanced predictor consistently achieved higher IPC values, with the largest improvements occurring in branch-heavy workloads such as `compression` and `coremark_im`. For example, the IPC of `compression` increased from 0.31 to 0.51, representing a substantial performance gain due to fewer pipeline flushes and more effective speculative execution. Similarly, `coremark_im` improved from 0.27 IPC to 0.43 IPC as the predictor reduced the high misprediction rates observed in the baseline design. Some workloads, such as `aes_sha`, saw smaller improvements because they contain more regular computation patterns and fewer difficult control-flow decisions. Overall, the dynamic predictor greatly reduced wasted execution on wrong-path instructions and allowed the out-of-order engine to better utilize available instruction-level parallelism. However, these performance gains came with important tradeoffs in both area and power consumption. The predictor required several hardware tables, including saturating-counter arrays for GShare and Bimodal prediction, chooser tables for tournament selection, and a tagged Branch Target Buffer (BTB) for storing predicted branch destinations. These structures increased total storage requirements and introduced additional combinational logic and SRAM accesses during instruction fetch, increasing frontend power usage and contributing noticeably to overall processor area.

3.3.2 RAS

Design The Return Address Stack is a hardware structure that keeps track of all returns for jumps. In RISC-V, returns are for jal and jalr instructions. The RAS module keeps two parallel copies of a fixed-depth ring buffer. One is a speculative stack updated as instructions are fetched and enqueued, and the other is a committed stack updated only when the commit stage confirms a call. On a pipeline flush (branch mispredict), the speculative pointer and array are restored from the committed snapshot so wrong-path fetches do not corrupt architectural RAS behavior. In our fetch_stage.sv, when a return is recognized and the RAS has a valid top entry, we override the branch predictor's target with the ras_top_addr signal and force a taken prediction, so returns do not rely on the BTB alone.

Testing To show the benefits of the RAS, here are the IPC values for all the benchmarks.

Benchmark	IPC (Tournament Predictor)	IPC (Tournament Predictor + RAS)
coremark_im	0.431	0.469
fft	0.452	0.498
mergesort	0.347	0.431
aes_sha	0.358	0.270
compression	0.514	0.505
image	0.316	0.364

Table 2: IPC comparison between the tournament branch predictor and the tournament predictor with a Return Address Stack (RAS).

Performance Analysis Adding the Return Address Stack (RAS) improved overall processor performance by improving the prediction accuracy of function return instructions. The RAS stores return addresses during function calls and predicts the correct return target during `jalr` return instructions, reducing costly frontend pipeline flushes and wrong-path execution. Several benchmarks showed noticeable IPC improvements after adding the RAS. For example, `mergesort` improved from 0.347 IPC to 0.431 IPC, while `fft` improved from 0.452 IPC to 0.498 IPC. The largest gains generally occurred in workloads with frequent nested function calls and returns, where return prediction accuracy is especially important.

However, not all benchmarks improved. For example, `aes_sha` experienced a decrease in IPC despite achieving a lower branch misprediction rate. This demonstrates that processor performance is influenced by other bottlenecks such as memory dependencies, ROB pressure, and reservation station stalls in addition to branch prediction accuracy. The RAS also introduced additional hardware structures, increasing frontend area and power consumption. Despite these tradeoffs, the RAS was still highly beneficial overall because it significantly reduced return-address mispredictions and improved frontend fetch efficiency across most workloads.

3.3.3 SRAM Branch Prediction (GShare + BTB)

Design As mentioned above, our area massively increased since the branch prediction structures were all implemented with flip-flops. To combat this, we removed some branch prediction mechanisms such as the Bimodal predictor (hence also getting rid of the tournament predictor), and replaced the branch prediction module with an SRAM Gshare predctor and BTB. The Gshare branch predictor and BTB work together to determine the direction and target of branches. The word index from the fetch PC is XORed with the global history register to index a SRAM-backed PHT of 2-bit saturating counters. The "taken" direction is the counter's MSB, and a real taken prediction with a target only happens when that agrees with a tagged BTB hit which is indexed from the PC. The GHR advances on fetch with the predicted outcome and is corrected on committed branch mispredicts using saved metadata. Committed conditional branches train the PHT at the saved index, and taken branches update the BTB, with small caches hiding SRAM latency. The RAS is a separate module, instantiated in the fetch stage and driven at commit. It keeps parallel speculative and architectural stacks and pointers. On enqueue, a call (JAL/JALR with rd == x1) pushes pc+4 speculatively, and a return (JALR with rd==x0, rs1==x1, imm==0) pops when the top is valid. Commit pushes the link address (commit_pc + 4) for calls and pops for returns so the commit stack tracks the true RAS. On flush, the speculative stack and contents are restored from the commit snapshot. In fetch_stage, when a return has a valid RAS top, RAS overrides the branch predictor's taken bit and target for PC and for the metadata enqueued with the instruction.

Testing This is a comparison of the IPC values we received with our branch predictor module made of SRAMs, as compared to the module with flip flops and the tournament logic.

Benchmark	IPC (Tournament + RAS)	IPC (SRAM GShare + BTB)
coremark_im	0.469	0.440
fft	0.498	0.455
mergesort	0.431	0.461
aes_sha	0.270	0.404
compression	0.505	0.523
image	0.364	0.340

Table 3: IPC comparison between the tournament predictor with RAS and the SRAM-backed GShare + BTB predictor.

Performance Analysis Although the SRAM-backed predictor greatly improved our area, it also introduced a noticeable reduction in IPC across several workloads compared to the earlier FF-based tournament predictor with RAS. Part of this performance difference comes from the added access latency and reduced flexibility of SRAM structures, which can negatively impact prediction timing and frontend throughput. Additionally, the SRAM-backed

predictor was implemented later in development alongside other architectural changes such as the split load-store queue and a prefetcher, meaning the IPC results are influenced by multiple interacting features rather than branch prediction alone. Even with these differences, the performance reduction was still evident across several benchmarks. Despite this trade-off, implementing the SRAM-backed predictor was an important architectural improvement because it demonstrated the importance of balancing performance with realistic hardware costs such as area and power. The redesign showed that it is possible to significantly reduce hardware overhead while still maintaining performance levels relatively close to the earlier high-performance baseline designs.

3.3.4 Stride Prefetcher

Design We implemented a stride prefetcher (`stride_prefetch.sv`) for the D-cache to reduce miss penalties for memory-bound loops with regular access patterns. The prefetcher maintains an 8-entry Reference Prediction Table (RPT), indexed by the low bits of the load PC (`obs_pc[RPT_IDX_BITS+1:2]`). Each RPT entry stores a valid bit, a 2-bit FSM state, the previous cache-line address (`prev_cl`), and the observed stride in cache lines (`stride_cl`). All tracking is done at cache-line granularity (27-bit `addr[31:5]`) to avoid aliasing within a line.

The FSM has four states: `INIT`, `TRANSIENT`, `STEADY`, and `NO_PRED`. On every D-cache load hit, the prefetcher computes the current stride as `obs_cl - prev_cl` and checks whether it matches the stored stride. When a new PC is first observed the entry is allocated in `INIT`. From `INIT`, a stride match promotes the entry to `STEADY`; a mismatch updates the stored stride and moves to `TRANSIENT`. From `TRANSIENT`, a match again reaches `STEADY`; a second mismatch moves to `NO_PRED`. `STEADY` entries stay steady on matches and fall back to `TRANSIENT` on a mismatch. `NO_PRED` entries recover to `TRANSIENT` on the next match. Prefetches are issued in `INIT`, `TRANSIENT`, and `STEADY` whenever a stride match is observed; `NO_PRED` suppresses prefetching entirely.

The prefetch target address is computed as:

$$\texttt{pf\_addr} = (\texttt{obs\_cl} + \texttt{PREFETCH\_DIST} \times \texttt{stride\_cl}) \ll 5$$

with `PREFETCH_DIST`=2 (two cache lines ahead). Only one prefetch is outstanding at a time (`pf_valid` register). The prefetch request is gated by `demand_active` so that an in-flight demand fill always takes priority over the speculative prefetch, and a zero stride is suppressed to avoid issuing redundant requests to the same line.

Testing The prefetcher was tested by comparing IPC before and after implementation on d-cache intensive memory programs. After implementation and associated bug fixes, we looked at the performance improvement for our benchmarks and compared it to the area. We found that our prefetcher implementation took around 4k area, which we deemed acceptable for the performance increase shown below.

Performance Analysis The following tables compares IPC of our final processor with and without the stride prefetcher. The stride prefetcher provides the largest gains on memory-intensive programs with regular access patterns (e.g., `image/compression`), where the RPT quickly reaches `STEADY` state and consistently prefetches two cache lines ahead. Sequential workloads with few strided loads show smaller benefit, as the D-cache is rarely the bottleneck.

Benchmark	IPC (no stride prefetch)	IPC (stride prefetch)
`coremark_im`	0.48	0.48
`fft`	0.49	0.49
`mergesort`	0.48	0.48
`aes_sha`	0.39	0.40
`compression`	0.60	0.68
`image`	0.35	0.40

Table 4: IPC comparison with and without the stride prefetcher.

3.3.5 Next-Line Instruction Prefetcher

Design We implemented a next-line instruction prefetcher (`next_line_prefetch.sv`) that sits between the fetch stage and the I-cache to hide sequential cache-line fill latency. The module proxies all demand fetches to the cache while also issuing speculative prefetches for the cache line immediately following the one currently being consumed.

The prefetcher tracks two pieces of state: `pf_waiting` (a prefetch is in flight) and `pf_completed_valid`/`pf_completed_line` (the most recently prefetched line address). A prefetch is needed (`need_pf`) whenever the cached prefetch result does not match `fetch_line_base + 32`. A prefetch is actually issued (`start_pf_combo`) only when the line buffer is serving a hit and therefore no demand request is active (`fetch_rmask == 4'b0`), no prefetch is already pending, and `need_pf` is true.

The cache address multiplexer into the I-cache uses a priority scheme. A flush or branch-taken event immediately drives `cache_addr` to the target cache-line base and asserts `cache_rmask`, cancelling any in-flight prefetch and treating the redirect target as the new demand. When a real fetch demand arrives (`fetch_rmask != 0`), it takes the cache port unconditionally and `pf_waiting` is cleared, since the demand fill will warm the cache naturally. Only when the port is otherwise idle does the prefetcher claim it to issue or continue a speculative fill. When the cache responds to a prefetch with no concurrent demand, `pf_completed_valid` is set, recording the prefetched line address so that the condition `need_pf` remains false until the fetch PC moves to the next line, at which point a new prefetch is automatically triggered.

Testing The testing procedure for the nextline prefetcher was near identical to that of the stride prefetcher. We noted that the required area was 800 units, very small compared to the rest of the processor. This meant that any noticed performance increases were basically free in terms of power and area consumption. Although the performance improvements are

not massive, they still allowed us to edge over the required IPC values for the benchmark programs.

Performance Analysis The following table compares IPC with and without the next-line prefetcher. The benefit is really only helpful on sequential instruction-heavy workloads where the fetch stage frequently crosses cache-line boundaries, causing I-cache misses that stall the entire front end. The prefetcher eliminates most of these stalls by filling the next line during line-buffer-hit cycles. Branch-heavy code (such as these benchmarks) sees smaller gains because mispredictions redirect the PC to non-sequential targets, invalidating the speculative fill. This feature would have much more of a noticable impact when the CPU runs large sequential workloads.

Benchmark	IPC (no next-line prefetch)	IPC (next-line prefetch)
`coremark_im`	0.48	0.48
`fft`	0.49	0.49
`mergesort`	0.48	0.48
`aes_sha`	0.38	0.40
`compression`	0.68	0.68
`image`	0.39	0.40

Table 5: IPC comparison with and without the next-line instruction prefetcher.

3.3.6 Split Load-Store Queue (LSQ)

Design Our baseline processor used a unified memory issue structure for both loads and stores. While simple, this introduced serialization between independent memory operations. To improve memory-level parallelism and reduce structural hazards, we implemented separate Load Queue (LQ) and Store Queue (SQ) structures. The design uses: A dedicated `new_load_queue` and a dedicated `store_queue`.
The load queue tracks speculative in-flight loads and supports oldest-ready-first scheduling using ROB-relative age computation:

$$rob\_age = rob\_idx - rob\_head$$

This preserves memory ordering while allowing out-of-order issue among independent loads. The load queue also performs dependency checks, operand wakeup through the CDBs, and blocking detection for unresolved older stores.

The store queue maintains stores in program order using a circular FIFO with head/tail pointers. Stores only commit to memory when both address and data operands are ready and the store reaches the SQ head. To simplify ordering logic, unresolved older stores conservatively block younger loads when the store address is unknown:

```
if (st_older && !store_queue_entries[s].rs1_ready)
```

```
blocked = 1'b1;
```

Although this reduces memory-level parallelism, it avoids memory-ordering violations and simplifies implementation. Finally, we used relatively small queue sizes (`LOAD_QUEUE_DEPTH=4`, `STORE_QUEUE_DEPTH=4`) to balance performance and timing.

Testing The split LSQ was tested by writing a small test assembly file and verifying in Verdi if loads that could go out-of-order, actually executed out-of-order as long as said loads were older than the oldest store. Once the core mechanic worked, we tested with increasingly more complex test programs, focused on load and store instructions. After an initial implementation, we also tried to experiment with different load and store queue depths to optimize area while maintaining good performance.

Performance Analysis We compare the split load/store queue design against the original unified LSQ organization. The unified LSQ achieves slightly higher IPC due to more flexible sharing of queue entries between loads and stores, improving memory operation throughput. However, the split LSQ significantly reduces queue contention and structural hazards, lowering LSQ full stall cycles and backend pressure.

Metric	**Unified LSQ (CP3)**	**Split LSQ**
Total Cycles	1,119,566	1,165,279
Coremark IPC	0.272	0.261
LSQ Full Stall Cycles	553	1,334
Load Blocked Cycles	32,118	28,465
Load Dispatches	103,180	112,041
Store Dispatches	40,025	40,060
Load Requests	118,851	114,347
Store Requests	84,885	93,965

Table 6: Performance comparison between unified and split LSQ organizations.

Although this only looks at Coremark, the other programs that we ran displayed similar results. Compared to the unified LSQ, the split LSQ reduced load-blocked cycles by approximately 11.4% and increased load dispatch throughput by 8.6%, indicating improved load-side scheduling efficiency. However, overall IPC decreased by roughly 4.0% due to reduced queue utilization flexibility and increased LSQ-full stalls caused by static queue partitioning.

The split LSQ benefits load-heavy and memory-level-parallel workloads with many independent reads, such as graph traversal or sparse memory accesses, by reducing contention between loads and stores. However, it performs less effectively on store-heavy or mixed-memory workloads where fixed queue partitioning can create artificial bottlenecks and reduce overall throughput.

Overall, the unified LSQ provides better throughput and IPC by dynamically utilizing queue capacity across both loads and stores, while the split LSQ reduces contention and offers cleaner separation between memory operation types.

3.3.7 Load-Store Forwarding

Design To improve memory performance, we implemented load-store forwarding from the Store Queue (SQ) to the Load Queue (LQ). This allows loads to bypass the cache and obtain data directly from older in-flight stores, reducing unnecessary serialization and load latency.

The forwarding logic searches the SQ for the youngest older matching store:

```
if (st_older &&
    store_queue_entries[s].dmem_addr == queue[i].dmem_addr &&
    |(store_queue_entries[s].dmem_wmask &
      queue[i].dmem_rmask))
```

Forwarding occurs only when:

- The store is older than the load
- Store and load addresses match
- Byte masks overlap
- Store data is ready
- The store fully covers the load request

When forwarding succeeds, the load bypasses the cache and completes directly through the memory CDB. Forwarded data is formatted correctly for byte, halfword, and full-word loads. To simplify correctness, partial forwarding cases were conservatively blocked:

```
if (found_fwd && (!best_rs2_ready || !best_covers))
    blocked = 1'b1;
```

This avoided complex merging logic between multiple stores but reduced potential memory-level parallelism.

Each load performs associative comparisons across all SQ entries, including address matching, ROB-age ordering, and byte-mask overlap checks. As SQ depth increases, forwarding logic becomes more expensive in timing and FPGA resource utilization.

Testing We tested load-store forwarding in a similar way to the split-LSQ. We wrote a simple test assembly file that had some store and load instructions that share the same address. Then we verified in Verdi that there was a corresponding load forward hit and the data from the correct store was shared to the load. Once the core mechanic worked, we tested with increasingly more complex test programs, focused on load and store instructions.

Performance Analysis We compare the processor with and without load-store forwarding enabled. While overall IPC is similar, forwarding improves memory system efficiency by reducing load-to-store dependency penalties and increasing the effectiveness of in-flight memory resolution. In particular, forwarding reduces store buffer pressure and enables earlier satisfaction of dependent loads without accessing the data cache.

Metric	With Forwarding	No Forwarding
Coremark IPC	0.479	0.488
Load blocked by stores (cycles)	84,216	91,978
Store-to-load forward events	701	0
LSQ store-full stalls (cycles)	25,423	17,941
Load-forward hits (PCSB)	127	833

Table 7: Microarchitectural impact of load-store forwarding.

Although this only looks at Coremark, the other programs that we ran displayed similar results. Load-store forwarding reduced load dependency stall cycles by approximately 8.4% by allowing dependent loads to receive data directly from in-flight stores without accessing the cache. Although overall IPC changed by only about -1.8%, forwarding improved effective memory latency handling for dependent memory access patterns.

Load-store forwarding is most beneficial for producer-consumer memory patterns, stack-heavy code, and workloads with short store-to-load reuse distances, where dependent loads frequently access recently stored data. The feature provides less benefit for streaming or read-dominated workloads with few store-load dependencies, and conservative handling of partial forwards may limit performance on byte-granular or misaligned memory accesses.

Overall, although IPC remains comparable, forwarding reduces load dependency stalls and enables direct communication between stores and dependent loads, improving effective memory latency handling in dependent access patterns.

3.3.8 Post Commit Store Buffer

Design To prevent stores from stalling ROB retirement, we implemented a Post-Commit Store Buffer (PCSB) that decouples committed stores from D-cache write completion. Once a store commits architecturally, it is inserted into the PCSB and the ROB may continue retiring younger instructions without waiting for the cache response.

The PCSB is implemented as a circular FIFO parameterized by `DEPTH`. Each entry stores: the store address, store write mask, and store data.

A small FSM serializes writes into the D-cache:

- `S_IDLE`: wait for pending stores
- `S_ISSUE`: send store request

- `S_WAIT`: wait for cache response

Stores remain buffered until the cache acknowledges completion:

```
if (dcache_resp && state == S_WAIT)
    head_ptr <= head_ptr + 1'b1;
```

To preserve correctness, the PCSB also supports store-to-load forwarding. Loads compare against all buffered stores older than them. If a matching store fully covers the load byte mask, data is forwarded directly from the buffer:

```
if (buf_addr[slot] == load_addr &&
    overlap == load_rmask)
begin
    pcsb_fwd_hit  = 1'b1;
    pcsb_fwd_data = buf_wdata[slot];
end
```

Finally, the buffer is flushed on pipeline squash or reset to prevent speculative stores from reaching memory.

Since this buffer sits between the load-store unit and the D-cache, it replaces the d-cache queue we had in place for CP3. This allows us to save a little more area by removing the previous queue and instead use the PCSB alongside several muxes to do the same work.

Testing We noticed in early testing that the ROB was often filling up for certain programs like aes_sha and fft. This issue did not improve even when the ROB size was doubled, leading us to conclude that store instructions at the head of the ROB were stalling the rest of the processor. The subsequent fix was the PCSB, so seeing the immediate IPC improvement made it an easy inclusion into our final CPU design. We played with the size of the FIFO depth, but found diminishing returns after a value of 8, since 8 consecutive store instructions were not super likely in the benchmarks.

Performance Analysis The PCSB absorbs committed stores into its 8-entry circular FIFO and drains them to the D-cache through a three-state FSM (`S_IDLE` → `S_ISSUE` → `S_WAIT`), hiding the cache-write latency from the commit path.

The gains are most pronounced on benchmarks with dense store sequences (`coremark_im`, `fft`, `mergesort`, `aes_sha`), where store-to-cache stalls at the ROB head were a primary bottleneck. The PCSB also performs store-to-load forwarding: loads that hit a buffered-but-not-yet-drained store receive the data directly from the PCSB rather than stalling until the store reaches the cache, which further reduces effective store-to-load latency.

Benchmark	IPC (no PCSB)	IPC (PCSB)
coremark_im	0.45	0.48
fft	0.45	0.49
mergesort	0.44	0.48
aes_sha	0.35	0.40
compression	0.67	0.68
image	0.38	0.40

Table 8: IPC comparison with and without store buffer.

4 Additional Observations

The two most significant implementation challenges were area and timing, and both required architectural trade-offs. After the full feature set was implemented, the design synthesized to over 900k cells, forcing us to greatly cut the sizes of our implementation structures. Each reduction was evaluated against the coremark IPC to ensure area savings did not greatly hurt the throughput of our processor. Timing presented an equally serious challenge, as the initial critical path ran combinationally from reservation station ready-bit evaluation through the issue selector and into the execution units, failing the 2.5 ns target by a significant margin. Registering the issue selector output in `issue_logic.sv` broke this path into two shorter cycles, and migrating the branch predictor from flip-flops to SRAM macros reduced flip-flop count, together enabling timing to successfully synthesize at 400 MHz. Once these constraints were met, the advanced features proved to be mutually helpful. The GShare + BTB predictor minimizes flush penalties, keeping the ROB and reservation stations consistently utilized; the next-line prefetcher eliminates I-cache stalls on the now-steady fetch stream; the stride prefetcher hid D-cache miss latency on the memory path; and the post-commit store buffer decoupled store writeback from ROB commit. These architectural improvements together helped us hit the required IPC benchmarks. Our final block diagram is shown below.

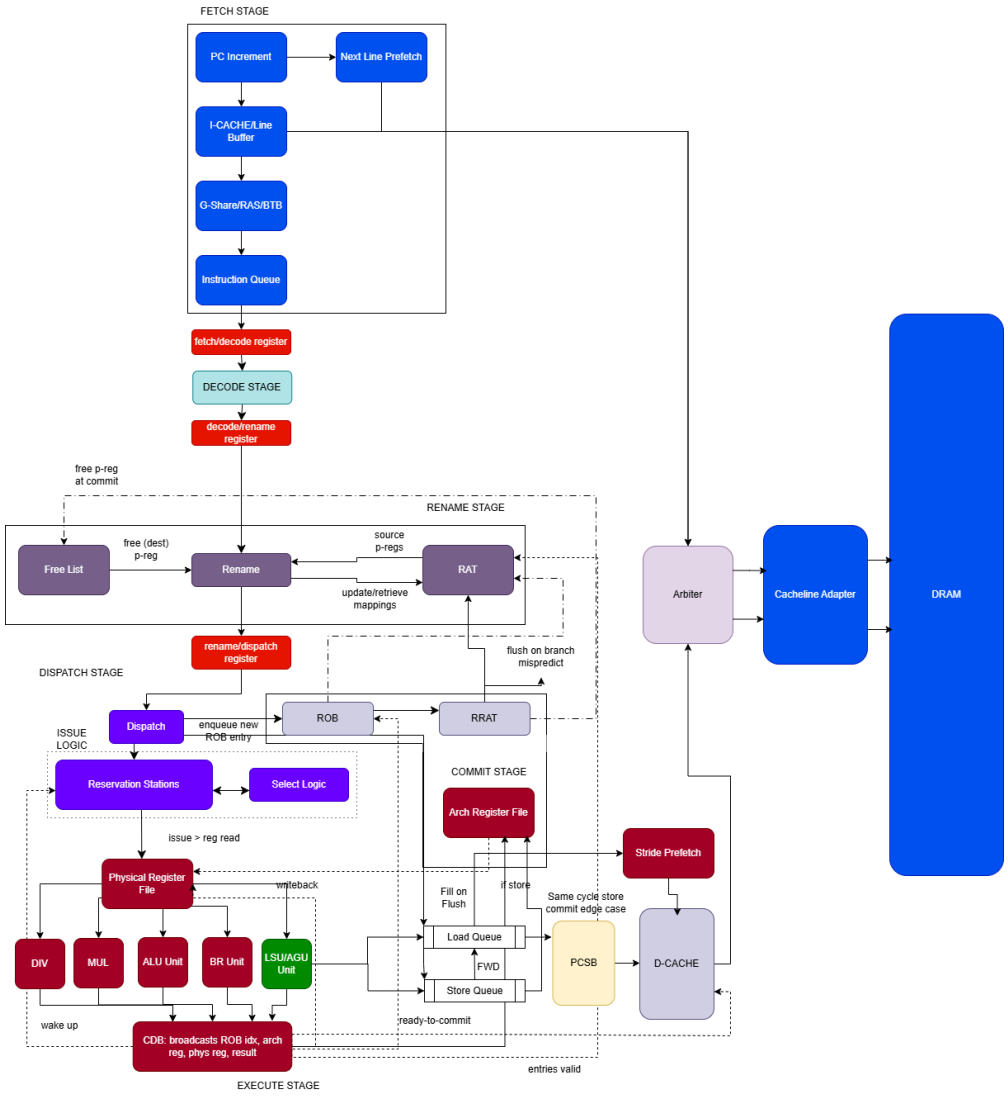

Figure 2: Advanced Feature block diagram.

4.1 Area Breakdown

The final design synthesizes around 290k μm^2. The table below shows the area contribution of the major hierarchical components reported by Synopsys.

Component	Area (μm^2)	% of Total
Execute Stage (total)	63,165.96	25.1%
Physical Register File	19,327.03	7.7%
Split LSQ	27,764.55	11.0%
Multiplier (DW_mult_pipe)	7,169.76	2.8%
Divider (DW_div_seq)	4,891.74	1.9%
D-Cache (32-set, 2-way)	37,501.87	14.9%
Branch Predictor (GShare+BTB)	26,704.52	10.6%
I-Cache (16-set, 2-way)	23,669.81	9.4%
Fetch Stage	12,382.03	4.9%
Dispatch Stage	1,747.35	0.7%
Decode Stage	1,280.26	0.5%
DRAM Arbiter	2,893.55	1.2%
Cacheline Adapter	3,087.20	1.2%
Remaining (ROB, rename, misc.)	8,780.90	3.5%
Total Cell Area	**294,611.44**	**100%**

Table 9: Hierarchical area breakdown of major CPU components

As shown above, the execute stage takes most of the allotted area, with the 48 physical registers and LSQ being a significant portion of the entire design. The caches and branch predictor similarly occupy a large amount of area due to the large amount of SRAM cells.

4.2 Power and Timing by Benchmark

Area constraints were the primary limiting factor on performance throughout this project. With the design already at 84% of the area budget, structures that most directly drive IPC—the ROB depth, reservation station counts, and physical register count—had to be reduced from their initial sizes to close area. The ROB was cut from 32 to 16 entries and physical registers from 64 to 48; both reductions narrow the out-of-order window and increase the frequency of full-ROB stalls, capping IPC on long-latency benchmarks. Table 10 reports the synthesis-estimated power and delay for each benchmark program.

Benchmark	IPC	Power (mW)	Delay
`coremark_im`	0.48	41	1.52
`fft`	0.49	41	5.93
`mergesort`	0.48	45	2.53
`aes_sha`	0.40	41	4.90
`compression`	0.68	46	1.58
`image`	0.40	41	2.94

Table 10: Benchmark Metrics

5 Conclusion

Our primary objective was to design an operational, high-performance RV32IM out-of-order processor within a 300k-cell area budget. The final processor correctly executes the full RV32IM ISA, passes RVFI formal checks, and achieves IPCs ranging from 0.40 to 0.68 across the various benchmarks. We were additionally able to implement a host of advanced features to help meet the baseline IPC requirements and also to learn what techniques modern processors use to push metrics. As we went through this design process we fixed countless memory bugs, race conditions, and logical errors that helped us better understand the lecture content of this course. At the very least, the (multiple) caffeine-fueled 3 am debugging sessions have ensured we know what happens on a branch mispredict. Overall, we are quite happy with the implementation and performance of our Out-Of-Order RISC-V processor.